210226

February 24, 2026

0.1 Functions

[1]:
```
## Nessacery imports for entire lab done here for easy management
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt
import hyperspy.api as hs
from scipy import optimize
```

[2]:
```
## Defining perodic function with a sine and cosine term, variables allow␣
↪manipulation of all characteristics,
## while the presence of a sin and cos allow variation of amplitude etc in a␣
↪period
def perodic(x,a,b,c,d,e,f,g):
    return (a*np.sin(b*np.deg2rad(x) + c) + (d*np.cos(e*np.deg2rad(x) + f)) + g)


## Definig perodic plot function to fit the perodic function to one slice of␣
↪data
def periodic_fit_plot(x_variable, y_variable,initial, plot = False):
## Using optimize.curve_fit to fit a perodic funtion to our x and y data,␣
↪intial values for parameters taken from input
    params_sin, params_sin_covariance = optimize.
↪curve_fit(perodic,x_variable,y_variable,p0=initial,maxfev=100000)

## Finding estimated uncertainties for each parameter
    errs = np.sqrt(np.diag(params_sin_covariance))

## creating a dummy variable for plotting purposes using linspace, parameters␣
↪of linspace ensure it varies from lowest to highest x value regarless of␣
↪order in array
    dummy = np.linspace(np.min(x_variable),np.max(x_variable),360)

## Calculating residuals
    difference = perodic(x_variable,*params_sin) - y_variable

## Calculating r squared for goodness of fit
```

```
## Squared sum of residuals
    ss_res = np.sum(difference**2)

## Squared sum of data - mean
    ss_tot = np.sum((y_variable- np.mean(y_variable))**2)

## Calculate r_squared per equation
    r_squared = 1 - (ss_res/ss_tot)

    ## Rounding frequencies, ensuring single values at 0/360
    params_sin[1] = np.round(params_sin[1], decimals=0)
    params_sin[4] = np.round(params_sin[4], decimals=0)
    if plot == True:
## Setting up subplots
        plt.subplots(2, 1, gridspec_kw={'height_ratios':␣
↪[2,1]},sharex=True,figsize=[10,6]) # height ratios make sizes right, sharex␣
↪makes plots share x axis
## Subplot 2 shows residuals
        plt.subplot(212)
        plt.plot(x_variable,difference,'or')
        plt.xlabel("Current (A)")
        plt.ylabel("Residuals")

        plt.grid()
        plt.tight_layout()
## Subplot 1 with fit and raw data
        plt.subplot(211)
        plt.scatter(x_variable, y_variable,label="Raw Data")
        plt.plot(dummy,perodic(dummy,*params_sin),color="r",label="Fitted␣
↪Function") ## using curvefit parameters and linear function to plot function
        plt.
↪fill_between(dummy,perodic(dummy,*(params_sin+errs)),perodic(dummy,*(params_sin-errs)),alph
↪2) ## highlights linear fit within 1 SD of both parameters, too small to see␣
↪for this data
        plt.plot([], [], ' ', label="R^2 = {0:.4f}".format(r_squared)) ##␣
↪displaying r squared value
        plt.grid()
        plt.legend()

    return(params_sin,errs)
```

[3]:
```
import numpy as np
import matplotlib.pyplot as plt

def sim_dataset(x_size,y_size,max_intensity, gaussian_noise, lam):
# Detector parameters
    nx, ny, nz = x_size, y_size, 360  # pixels x, pixels y, azimuthal bins
```

```
    # Azimuthal angle (0 to 2*pi)
    angles = np.linspace(0, 2*np.pi, nz)

    # Shifting sinsuisoid spatially
    phase_shift_x = np.linspace(0, 2*np.pi, nx)[:, np.newaxis, np.newaxis]
    phase_shift_y = np.linspace(0, 2*np.pi, ny)[np.newaxis, :, np.newaxis]

    # Amplitudes
    amp1 = np.random.randint(0,100)
    amp2 = np.random.randint(0,100)
    # Phases
    phase1 = np.random.randint(0,2*np.pi)
    phase2 = np.random.randint(0,2*np.pi)

    # Creating Base signal pattern
    signal_pattern = amp1 * np.sin(angles[np.newaxis, np.newaxis, :] +␣
↪phase_shift_x + phase_shift_y + phase1)**2 + amp2 * np.sin(2*angles[np.
↪newaxis, np.newaxis, :] + phase_shift_x + phase_shift_y + phase2)**2



    ## Generating noise to simulate realistic dataset
    # Gaussian noise
    gaussian_noise = np.random.randn(nx, ny, nz) * gaussian_noise
    # Poisson noise
    poisson_noise = np.random.poisson(lam=lam, size=(nx, ny, nz))

    # Combine signal and noise to create final data set
    detector_data = signal_pattern + gaussian_noise + poisson_noise

    # Clip to valid intensity range
    detector_data = np.clip(detector_data, 0, max_intensity)

    np.save("pixel_detector_azimuthal.npy", detector_data)

    return detector_data, signal_pattern
```

[4]:
```
def decomp_data(path, components, algo, output_dim = 0):
    raw_data = hs.load(path)
    raw_data_end = np.load(path)
    data_hs_func = hs.signals.Signal1D(raw_data.data)
    if output_dim > 0:
        data_hs_func.decomposition(algorithm=algo, output_dimension=output_dim)
    else:
        data_hs_func.decomposition(algorithm=algo)
    sc = data_hs_func.get_decomposition_model(components)
```

```
    decomposed_data = sc.data
    #data_hs_func.plot_explained_variance_ratio()
    return decomposed_data, raw_data_end
```

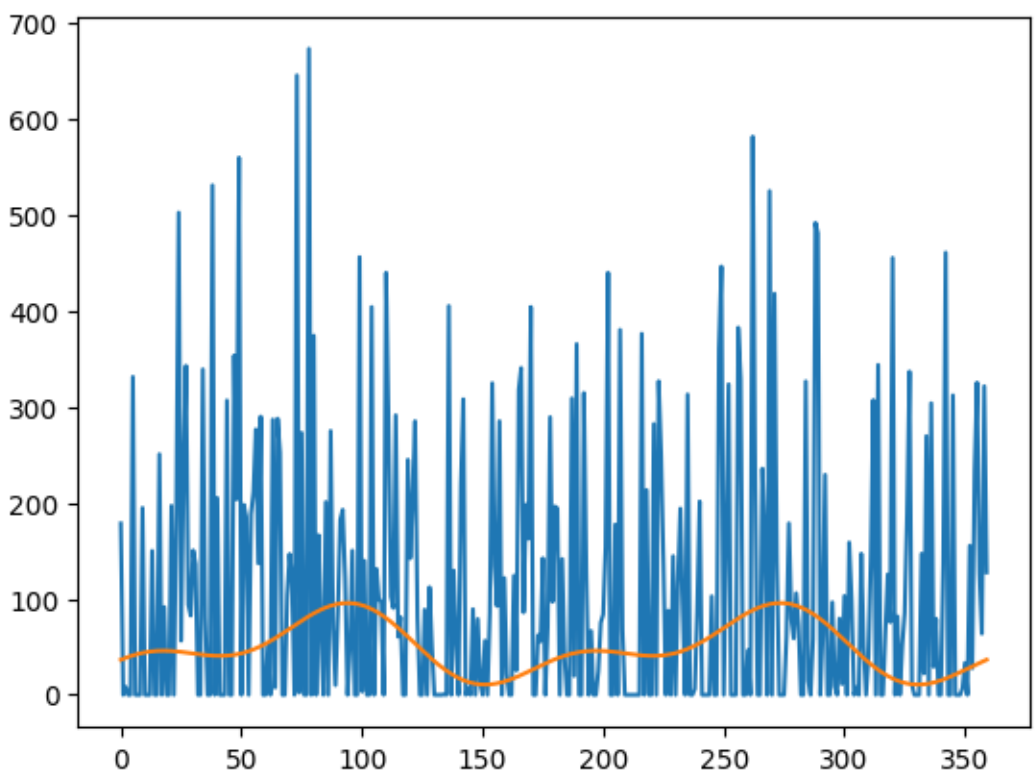
0.2 Data Processing

[5]:
```
## This notebook is the first testing of both the simulated dataset function,␣
↪then trying to estimate how good the algo is at recovering the ground truth
```

[6]:
```
## Simulating 100x100 dataset and saving for data processing
sim = sim_dataset(100,100,10000,200,10)
np.save("pixel_detector_azimuthal.npy", sim[0])
```

[7]:
```
## Plotting the noisy generated data and the underlying ground truth

plt.plot(sim[0][10,10,:])
plt.plot(sim[1][10,10,:])
```

[7]: [<matplotlib.lines.Line2D at 0x7fa4e5d5df40>]

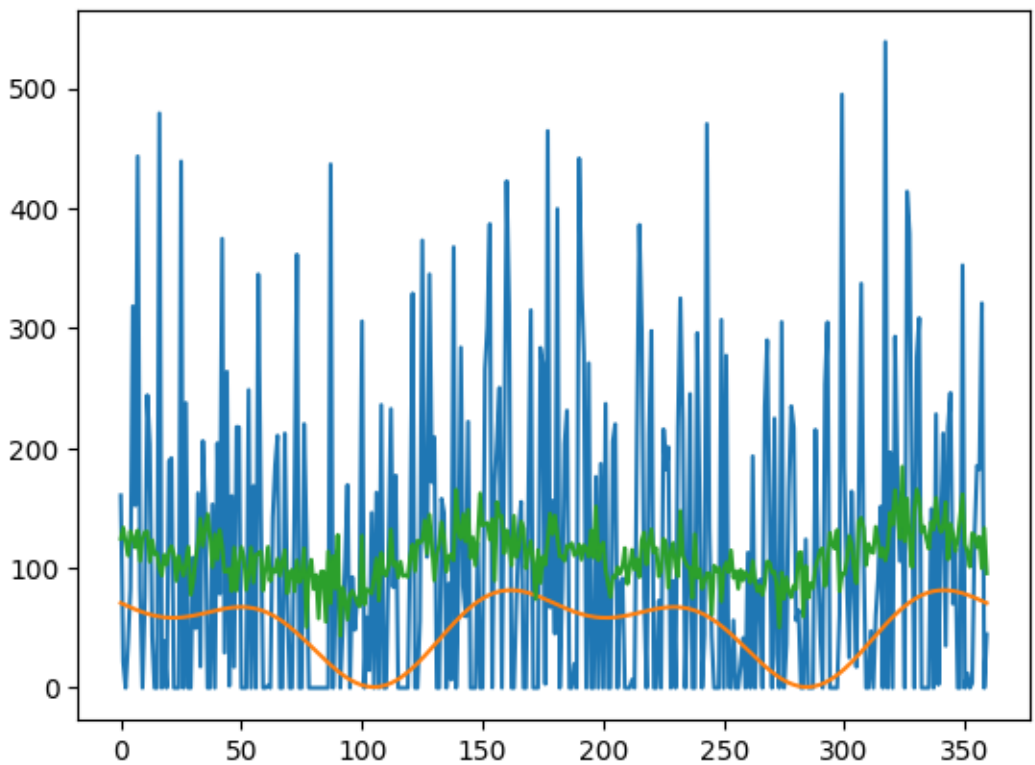


[8]:
```
## Decomposing the simulated data using SVD to investigate
decomp = decomp_data("pixel_detector_azimuthal.npy",5,"SVD")
```

```
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

[11]:
```
## Plotting ground truth, noisy data and denoised data

plt.plot(sim[0][20,20,:])
plt.plot(sim[1][20,20,:])
plt.plot(decomp[0][20,20,:])
```

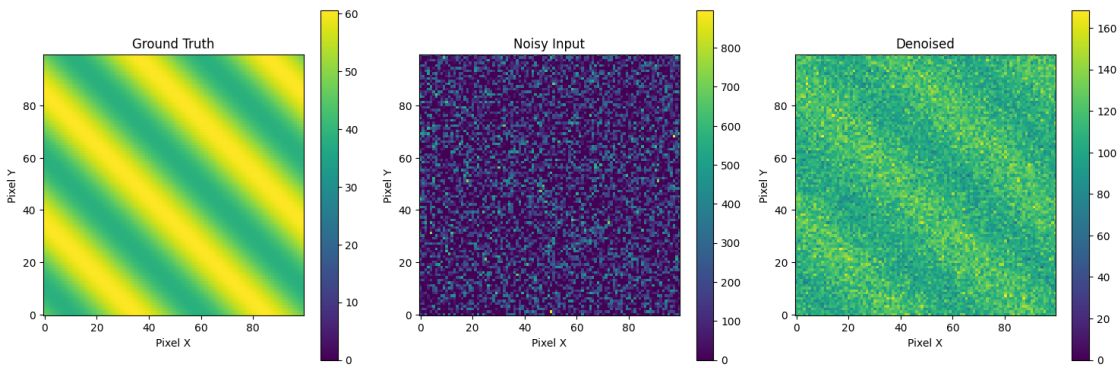
[11]:
```
[<matplotlib.lines.Line2D at 0x7fa43e55f3a0>]
```



We see that the denoised data approaches and appears similar the ground truth

[12]:
```
# Visually comparing slices

slice_index = 200  # azimuthal slice to visualize

fig, axes = plt.subplots(1, 3, figsize=(15, 5))  # 1 row, 3 columns

# Ground truth
im0 = axes[0].imshow(sim[1][:, :, slice_index], cmap='viridis', origin='lower',␣
↪vmin=0)
axes[0].set_title('Ground Truth')
plt.colorbar(im0, ax=axes[0])

# Noisy input
im1 = axes[1].imshow(sim[0][:, :, slice_index], cmap='viridis', origin='lower',␣
↪vmin=0)
axes[1].set_title('Noisy Input')
plt.colorbar(im1, ax=axes[1])

# Denoised output
```

```
im2 = axes[2].imshow(decomp[0][:, :, slice_index], cmap='viridis',␣
 ↪origin='lower', vmin=0)
axes[2].set_title('Denoised')
plt.colorbar(im2, ax=axes[2])

# Common labels
for ax in axes:
    ax.set_xlabel('Pixel X')
    ax.set_ylabel('Pixel Y')

plt.tight_layout()
plt.show()
```



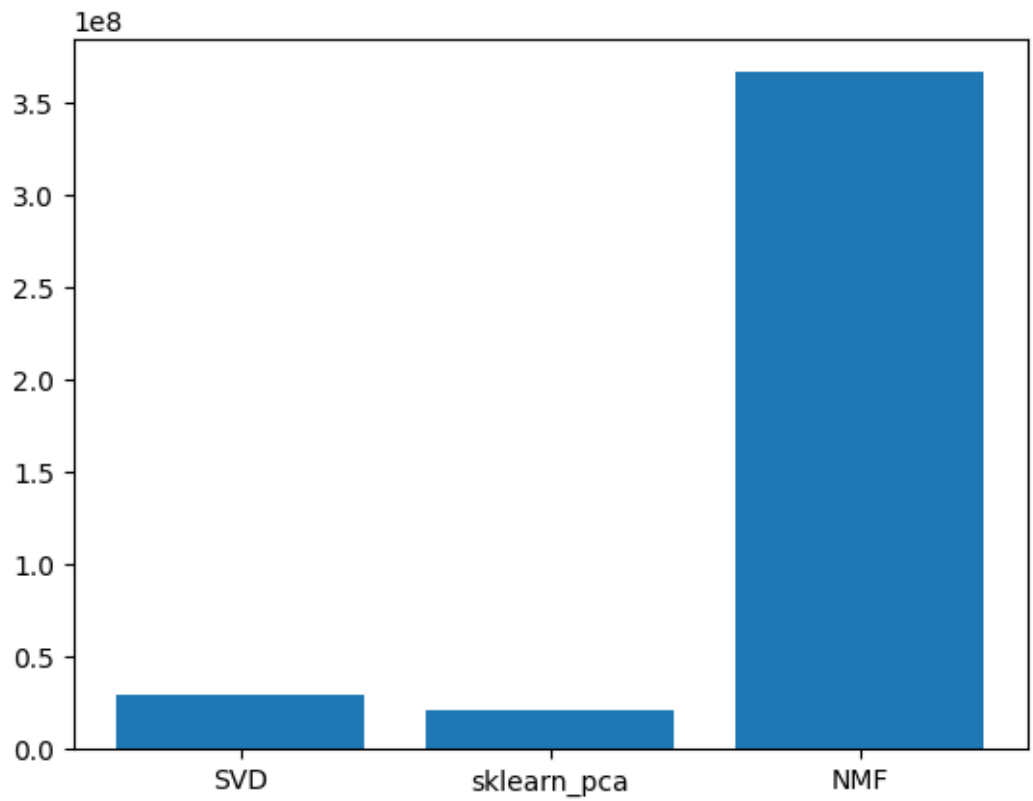
Once again, the algorithm is recovering the function from the noisy data, at least from a visual assessment of a slice

[13]:
```
## Running a selection of decomp algorithms and calculating the residuals of␣
 ↪the denoised data compared to the ground truth to assess how well it␣
 ↪recovering the underlying function
## Potentially can add more algorithms

algos = ["SVD", "sklearn_pca", "NMF"]


algos_res = []

for a in algos:
    decomp = decomp_data("pixel_detector_azimuthal.npy",5,a)
    residuals = 0
    for y in range(np.shape(decomp[1])[0]):
        for x in range(np.shape(decomp[1])[1]):
            residual = np.sum(decomp[1][y][x]-decomp[0][x][y])
            residuals += np.absolute(residual)
    algos_res.append(residuals)
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=sklearn_pca
  output_dimension=None
  centre=None
scikit-learn estimator:
PCA()
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=NMF
  output_dimension=None
  centre=None
scikit-learn estimator:
NMF(n_components=None)
```

[14]:
```
## PLotting resiudals of each algorithm

plt.bar(algos,algos_res)
#plt.ylim(ymin=140000000,ymax=165000000)
print(algos_res)
```

```
[29577677.205175888, 20977839.090951502, 366728833.2728777]
```



We see that sklearn_pca is the best performing algorithm on this dataset. SVD also performs well. NMF appears to be failing

Then, considered comparing the fits generated residuals to orignal function. Unsure as of now which is more appropriate.

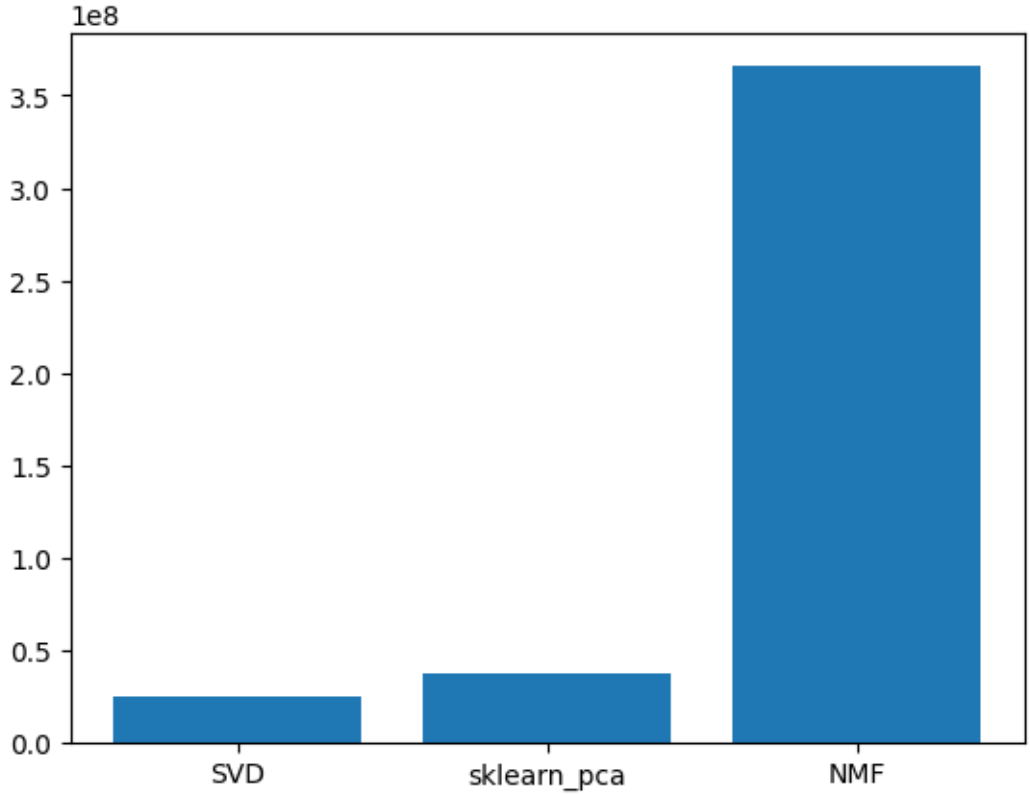
[16]:
```
## Running a selection of decomp algorithms and calculating the residuals of␣
↪the denoised data compared to the ground truth to assess how well it␣
↪recovering the underlying function
## Potentially can add more algorithms

algos = ["SVD", "sklearn_pca", "NMF"]

algos_res = []

for a in algos:
    decomp = decomp_data("pixel_detector_azimuthal.npy",5,a)
    residuals = 0
    for y in range(np.shape(decomp[1])[0]):
        for x in range(np.shape(decomp[1])[1]):
            fit = periodic_fit_plot(np.
↪linspace(0,len(decomp[1][x][y]),len(decomp[0][x][y])),decomp[1][x][y],[100,2,1,100,2,1,1])
```

```
            residual = np.sum(perodic(np.
↪linspace(0,len(decomp[0][x][y]),len(decomp[0][x][y])),*fit[0])-decomp[0][x][y])
            residuals += np.absolute(residual)
    algos_res.append(residuals)
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=sklearn_pca
  output_dimension=None
  centre=None
scikit-learn estimator:
PCA()
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=NMF
  output_dimension=None
  centre=None
scikit-learn estimator:
NMF(n_components=None)
```

[17]:
```
## PLotting resiudals of each algorithm

plt.bar(algos,algos_res)
#plt.ylim(ymin=140000000,ymax=165000000)
print(algos_res)
```

```
[25587359.7641526, 37750060.13154019, 365917330.757639]
```



We can see now that when comparing the generated fits to the ground truth SVD performs the best on this data set compared to sklearn_pca. NMF still remains far off the ground truth

Next steps for this is broadening the algorithms tested, refining the testing methods, maybe testing on multiple data sets / improving the simulation to be more realistic to experimental data.